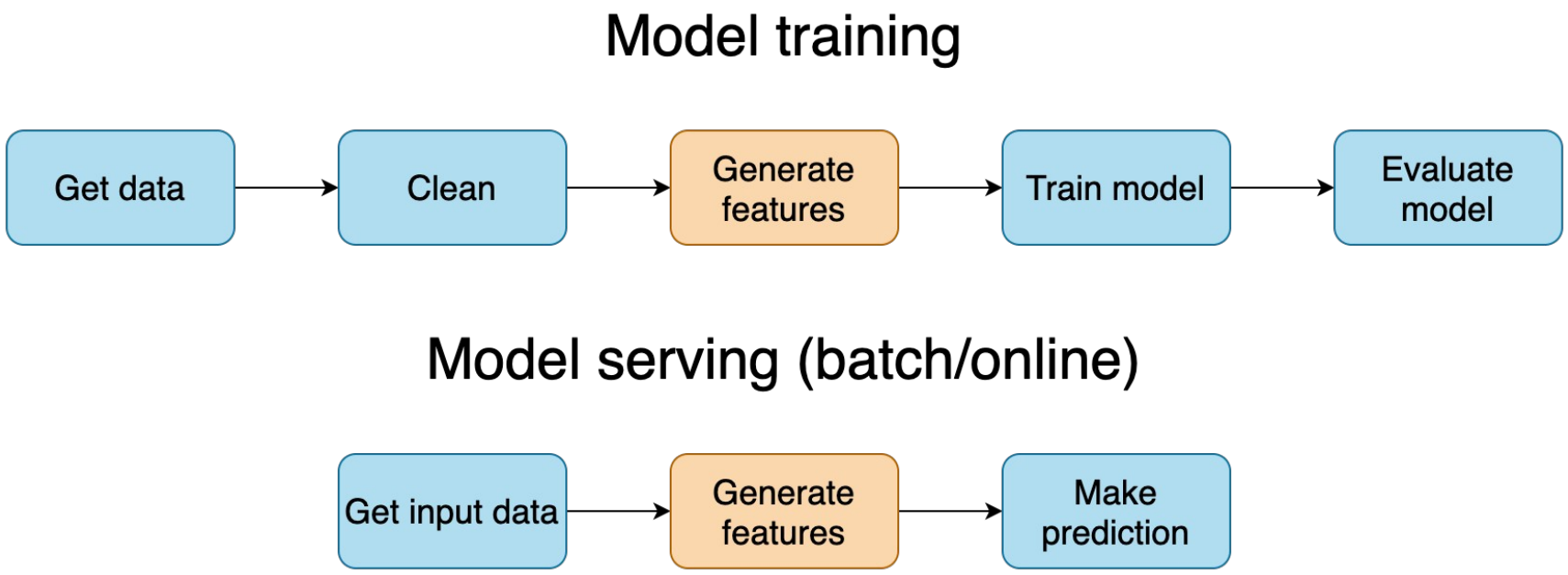
AI Infrastructure

Ioannis Papapanagiotou

Data Challenges

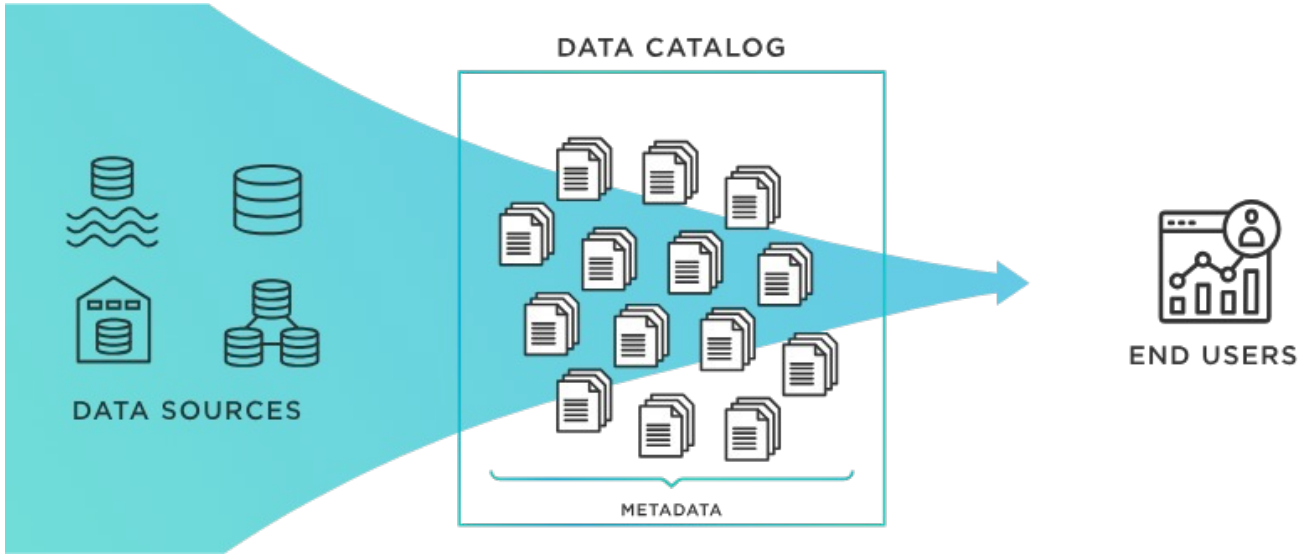
ML Model Pipeline



Data Challenge #1: Accessing the right raw data source

In order to develop any feature or model, a data scientist first needs to discover and access the right raw data source. This comes with several challenges:

- **Data discovery:** They need to know where the raw data actually exists. Data cataloging systems like Lyft's Amundsen are a great solution but not yet widely adopted. Often the data needed doesn't even exist in the first place and must first be *created* or logged.
- **Access approval:** Data scientists often have to jump through approval hoops to get access to data relevant for their problem.
- **Raw data access:** The raw data may be extracted as a one-time data dump that's out of date the moment it lands on their laptop. Or, the data scientist struggles her way through networking and authentication obstacles and is then faced with having to fetch the raw data using data source-specific querying languages.



Data Challenge #2: Building features from raw data

- Raw data can come from a variety of data sources, each with its own important properties that impact what kinds of features can be extracted from it. Those properties include a data source's support for **transformation types**, its **data freshness**, and the amount of available **data history**:

	Data Warehouse *(e.g. Snowflake)*	Transactional *(e.g. MySQL)*	Streams *(e.g. Kafka)*	Prediction Request Data
Data Quantity	Complete History	Recent History	Near Real-time	Real-time
Data Freshness	Low *(hours / days)*	Medium *(milliseconds – seconds)*	Medium *(milliseconds – seconds)*	High

Supported Transformations

Time Horizon

Large Scale Batch Aggregations	✓			
Small Scale Batch Aggregations	✓	✓		
Time Window Aggregations	✓	✓	✓	
Row Level Transformations	✓	✓	✓	✓

Type of Data Sources for Raw Data

- **Data warehouses** (such as Snowflake, AWS Redshift, Google Bigquery) tend to hold a lot of information but with low data freshness (hours or days). They can be a gold mine, but are most useful for large-scale batch aggregations with low freshness requirements, such as "number of lifetime transactions per user." → OLAP
- **Transactional data sources** (such as MongoDB, MySQL, Postgres) usually store less data at a higher freshness and are not built to process large analytical transformations. They're better suited for small-scale aggregations over limited time horizons, like the number of orders placed by a user in the past 24 hrs. → OLTP
- **Data streams** (such as Kafka) store high-velocity events and provide them in near real-time (within milliseconds). In common setups, they retain 1-7 days of historical data. They are well-suited for aggregations over short time-windows and simple transformations with high freshness requirements, like calculating that "trailing count over the last 30 minutes" feature described above.
- **Prediction request data** is raw event data that originates in real-time right before an ML prediction is made, e.g. the query a user just entered into the search box.
  - While the data is limited, it's often as "fresh" as can be and contains a very predictive signal. This data is provided with the prediction request and can be used for real-time calculations like finding the similarity score between a user's search query and documents in a search corpus.

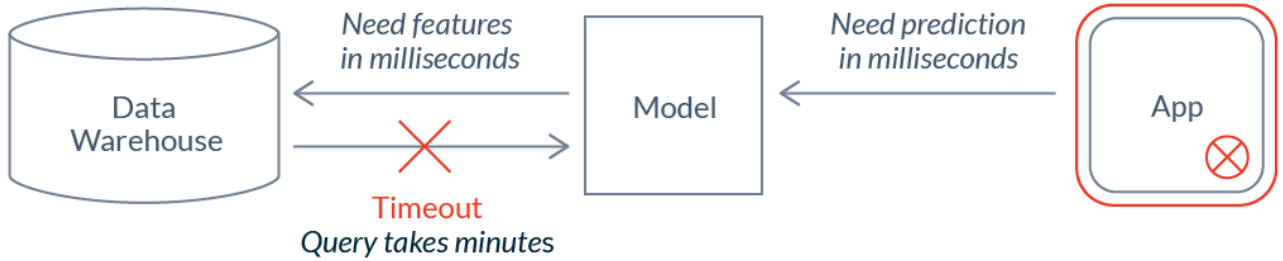
Data Challenge #3: Combining features into training data

Feature data needs to be combined to form training or test data sets. There are many things to look out for that can have critical implications on a model. Two notably tricky issues:

- **Data Leakage:** Data scientists need to ensure their model is training on the right information and that unwanted information is *not* "leaking" into the training data. This can be data from a test set, ground truth data, data from the future, or information that undoes important preparation processes like anonymization.
- **Time Travel:** One particularly problematic kind of leakage is data from the future. Preventing this requires calculating each feature value in the training data accurately with respect to a specific time in the past (i.e. time traveling to a point of time in the past). Common data systems aren't designed to support time travel, leaving data scientists to accept data leakage in their models or build a jungle of workarounds to do it correctly.

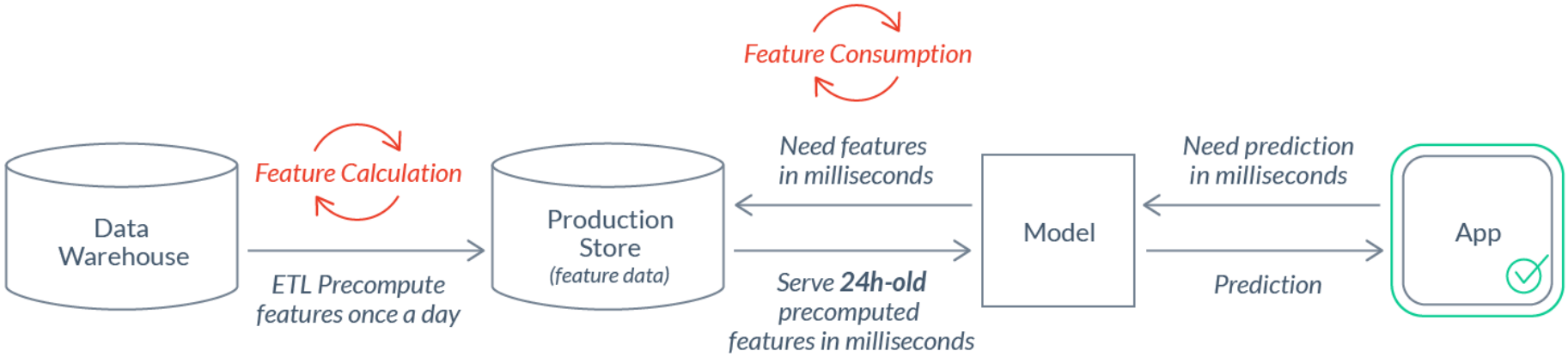
Data Challenge #4: Calculating and serving features in production

- After a model is deployed for real-time use cases, new feature data needs to be continually delivered to the model to power new predictions. Often, this feature data needs to be provided at low latency and high scale.
- How should we get that data to the model? Directly from the raw data source?
  - In many cases, that's not possible. It can take minutes, hours, or days to retrieve and transform data from a data warehouse, which is way too slow to be used for real-time inference:



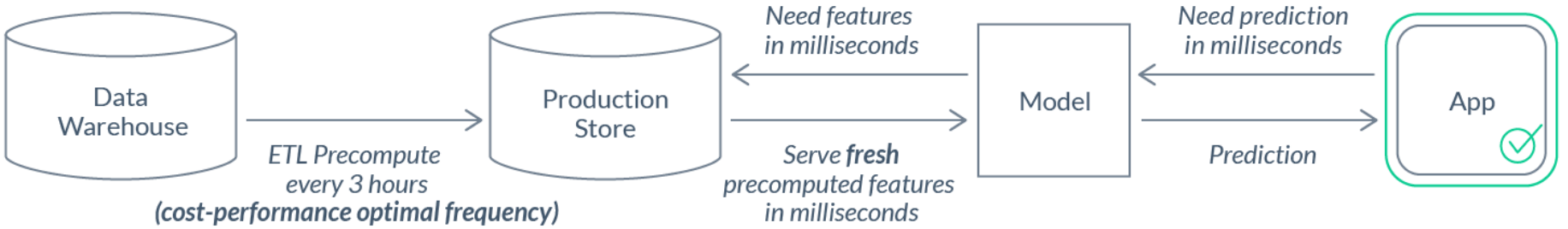
Data Challenge #4: Calculating and serving features in production (cont.)

- In those cases, feature calculation and feature consumption need to be *decoupled*.
- ETL pipelines are needed to *precompute* features and load them into a production data store that's optimized for *serving*. Those pipelines come with additional complexity and maintenance costs:



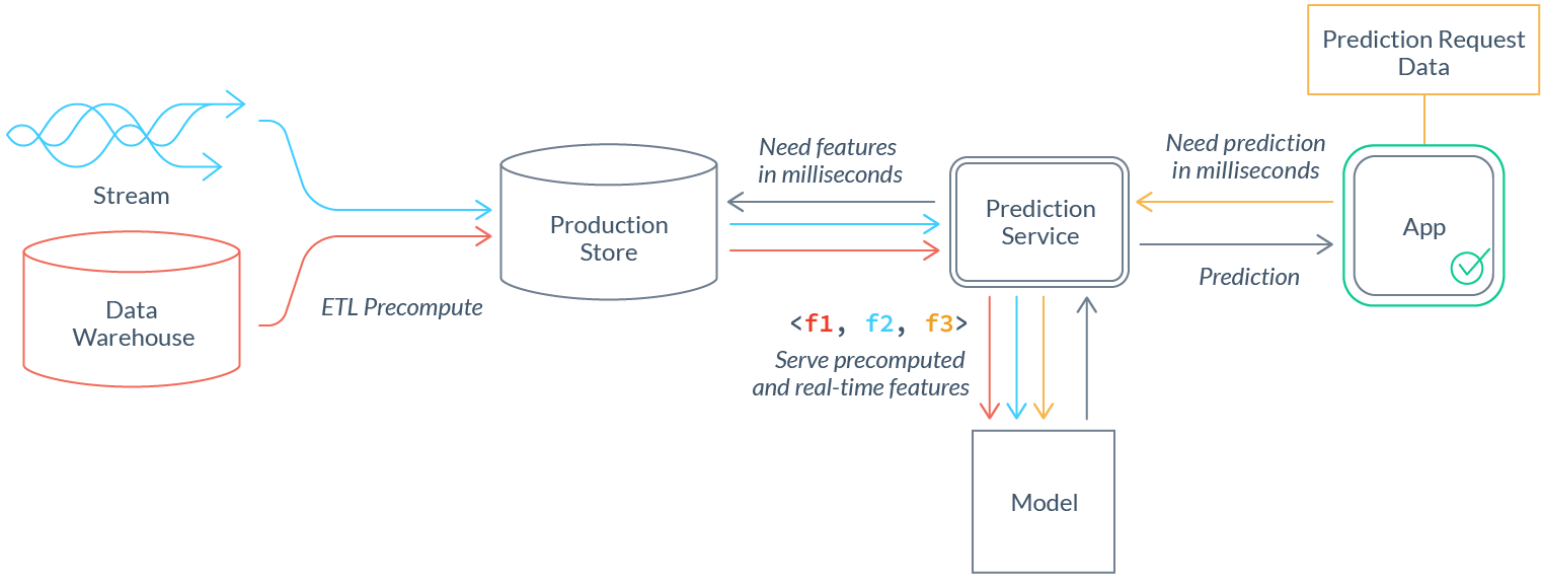
Finding the optimal freshness / cost-efficiency tradeoff

- Decoupling feature calculation from feature consumption makes freshness a primary concern.
- Often, feature pipelines can be run more frequently and result in fresher values, at the expense of added cost.
- The right trade-off here varies across features and use cases. E.g. for a 30-minute trailing window aggregation feature to be meaningful, it should be refreshed more frequently than a similar 2-week trailing window feature.



Integrating Feature Pipelines

- Complex challenges need to be solved to productionize features built on other types of data sources. Serious data engineering is typically required to coordinate these pipelines and integrate their outputs into a single feature vector:



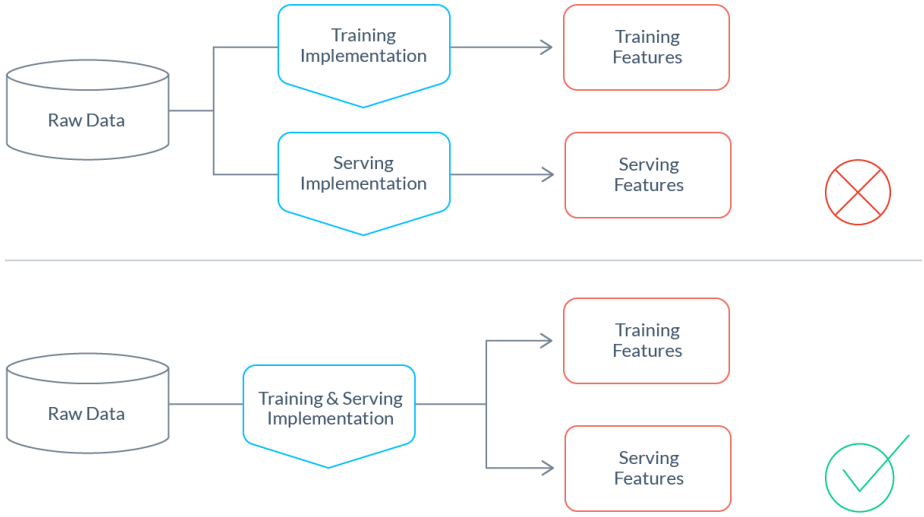
Avoiding training/serving-skew

Discrepancies between training and serving pipelines can introduce training/serving-skew. A model can act erratically when performing inference on data generated differently than the data on which it trained. Training/serving-skew can be very hard to detect and can render a model's predictions completely useless. Two common risks that are worth highlighting:

- **Logic discrepancies**
- **Timing discrepancies**

Logic Discrepancies

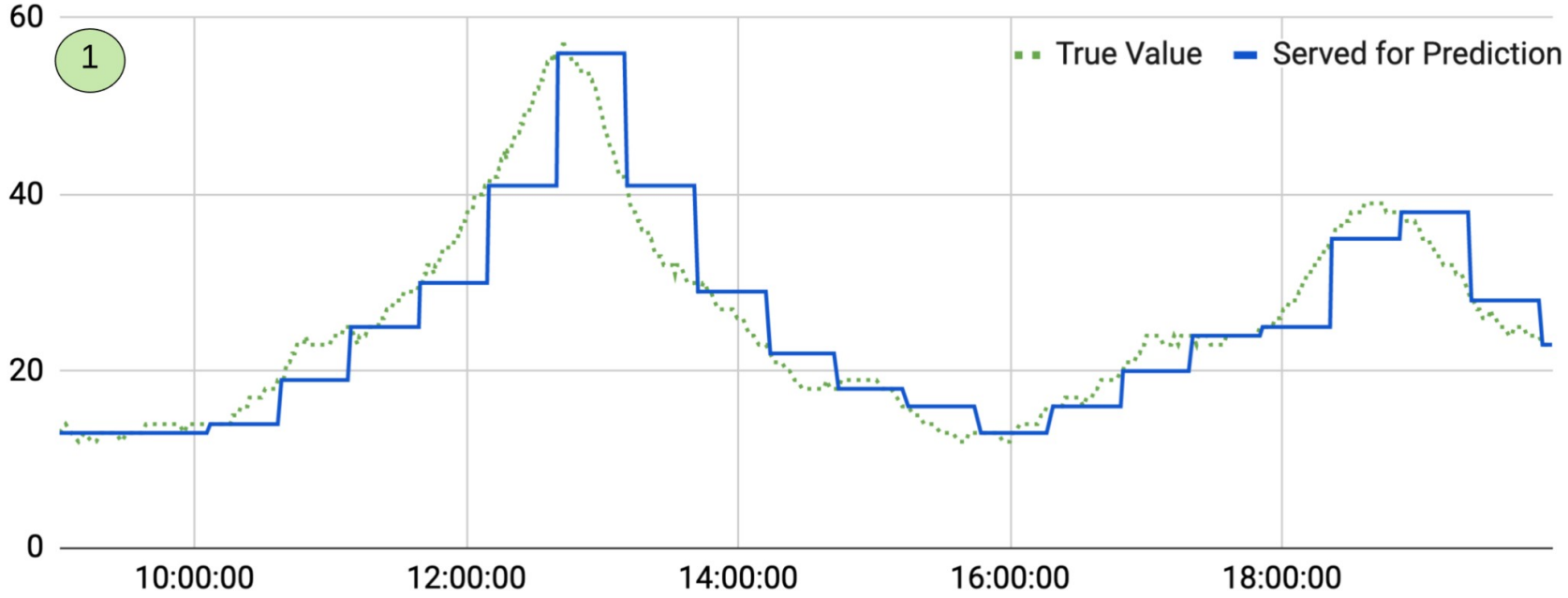
- When training and serving pipelines have separate implementations (as is the common practice), it's very easy for differences in transformation logic to be introduced.
  - Are Null values handled differently? Do numbers use consistent precision?
- Tiny discrepancies can have a huge impact. Best practice for minimizing risk of any skew is to reuse as much transformation code as possible across training and serving pipelines. This is so important that it's not uncommon to take on significant additional engineering effort to achieve this goal in the pursuit of saving endless future hours of painful debugging.



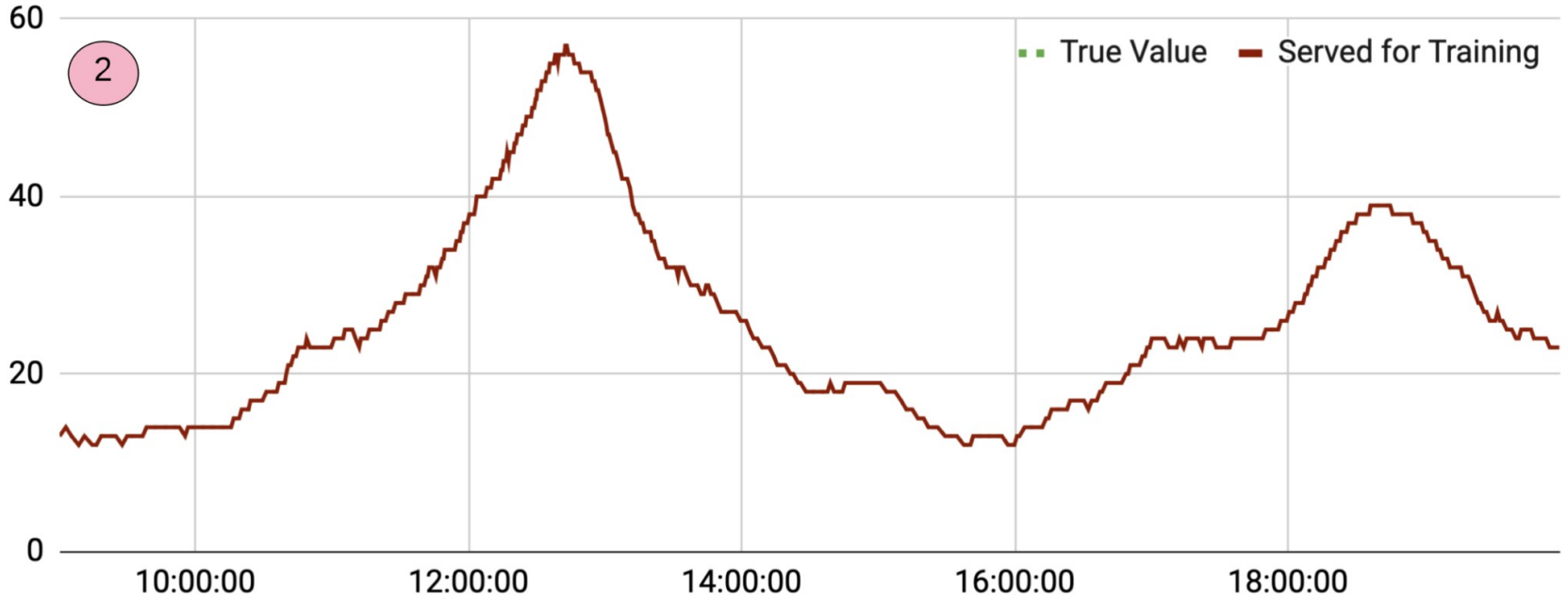
Time Discrepancies

- For a variety of reasons (like cost) features aren't always continuously precomputed. E.g., little additional signal is gained from recalculating a two-hour trailing count of orders for every customer and every second in production.
- In practice, feature values tend to be a few minutes, hours, or even days stale at inference time.
- A common mistake happens when this staleness is not reflected in the training data, leading the model to train on fresher features than it would receive in production.

Features used in production (with staleness)



Features used for training (no staleness)



Data Challenge #5: Monitoring features in production

- When an ML system breaks, it's nearly always due to a "data breakage." This can refer to many distinct problems for which monitoring is needed. A few examples:
- **Broken upstream data source:** A raw data source may experience an outage and suddenly provide no data, late data or completely incorrect data. It's all too easy for such an outage to go undetected. A data outage may have wide-reaching ripple effects, polluting downstream features and models. It's often very difficult to identify all affected downstream consumers. And even if they are identified, solving the problem via backfills is expensive and arduous.
- **Actionable feature drifts:** Individual features may start to drift. This could be a bug or it could be perfectly normal behavior that indicates that the world has changed (e.g. your customer's behavior may indeed have wildly changed based on a recent news break), requiring the model to be retrained.
- **Opaque feature subpopulation outages:** Detecting entire feature outages is fairly trivial. However, outages that affect only a subpopulation of your served model (e.g. all customers living in Germany) are much harder to detect.
- **Unclear data quality ownership:** Given that features may be derived from different upstream raw data sources, who is ultimately responsible for the quality of the feature? The data scientist who created the feature? The data scientist who trained the model? The upstream data owner? The engineer who did the production model integration? When responsibilities aren't clear, issues tend to remain unresolved much longer than necessary.